Introduction to TypeScript

TypeScript is a programming language developed by Microsoft in 2012. It is a superset of JavaScript, meaning any valid JavaScript code is also valid TypeScript code. TypeScript adds additional features such as static typing, interfaces, enums, and better tooling support.

Why TypeScript?

- Detects errors at compile time
- Improves code readability and maintainability
- Provides strong typing and IntelliSense
- \Makes large projects easy to manage
- Helps testers write reliable automation scripts

Why Not Only JavaScript?

1. Errors appear only at runtime
2. Difficult to refactor large projects
3. No type safety
4. Hard to maintain complex automation frameworks

TypeScript solves all these problems by adding compile-time checks and type safety.

History of TypeScript

- **TypeScript was introduced in 2012** by Microsoft to address scalability and maintainability issues in large JavaScript applications. It was designed to make JavaScript suitable for enterprise-level development.
- **Anders Hejlsberg led the development of TypeScript**, bringing strong typing concepts from languages like C# and Delphi. His goal was to improve developer productivity and code reliability.
- **TypeScript adds static typing on top of JavaScript,** allowing errors to be detected at compile time instead of runtime. This helped teams reduce bugs in complex projects.
- **TypeScript compiles (transpiles) to plain JavaScript,** ensuring compatibility with all browsers and JavaScript runtimes. This made adoption easy without changing existing JavaScript workflows.
- **Over time, TypeScript became widely adopted** in frameworks and tools such as Angular, Node.js, and Playwright.

Installing tsc :

npm install -g typescript // Global installation
tsc --version // Check version
tsc app.ts // Compile a single file
tsc --watch // Watch mode (auto recompile on changes)

What is tsc?

tsc stands for TypeScript Compiler.
It is a command-line tool provided by TypeScript that converts TypeScript code (.ts files) into JavaScript (.js files) and checks for type-related errors during compilation.

Why do we need tsc?

Browsers and Node.js cannot execute TypeScript directly.
tsc ensures that TypeScript code is validated, type-checked, and converted into standard JavaScript that can run in any environment.

What is a Transpiler?

- A transpiler converts code from one programming language to another.
- Mostly used to convert modern code into older, supported code.
- TypeScript is transpiled into JavaScript. • Browser cannot understand TypeScript directly. After transpiling, JavaScript is executed by browser or Node.js.



Type Inference

- Type Inference is a feature in TypeScript where the compiler automatically determines the data type of a variable, expression, or function return value based on the assigned value or context.
- This allows developers to write less code while still maintaining type safety.
- Type inference should be used only for primitive types.
-

Why Type Inference is Important?

- Reduces the need for explicit type annotations
- Keeps code clean and readable
- Maintains strong type safety
- Helps catch errors at compile time
- Improves developer productivity

Syntax:
let variable = value;
Examples :

```
// type inference
// it should be used for primitives.
let a = 10;
```

```
let b = "hello";
```

When Type Inference is Enough

- Variable is initialized immediately
- Function logic is simple
- Local or internal variables
- Short, readable code is preferred

Type Annotation :

- Type Annotation is the process of explicitly specifying the data type of a variable, function parameter, return value, or object in TypeScript.
- It tells the compiler what type of value is expected, helping catch errors at compile time.
- Use type annotation when you have non-primitive types.

Why Type Annotation is Needed :

- Prevents runtime errors
- Improves code readability
- Provides compile-time type safety
- Helps IDEs with auto-suggestions
- Makes code self-documenting

When to Use Type Annotation

- A variable is declared without an initial value
- Functions have complex return logic
- Writing reusable utilities or APIs
- Working in large automation frameworks

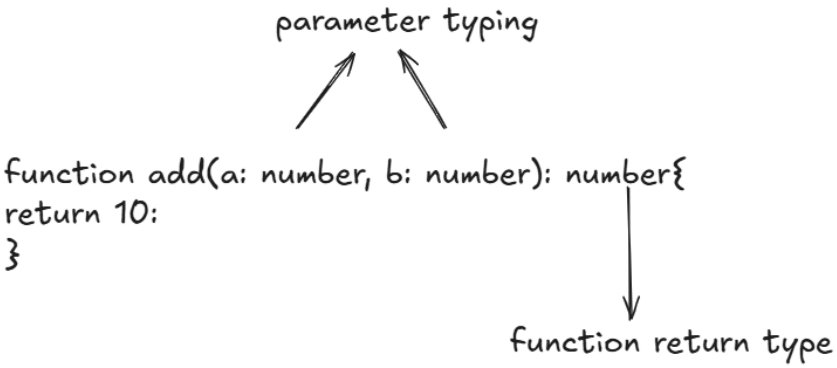
Syntax:
let variableName: type = value;

Examples :

```
// type annotation
// this type annotation should be used for non-primitives.
let c: number = 10;
let d: string = "";
```

Annotating using functions :

```
function demo(a: number, b: number): number {
  return a + b;
}
```



Annotating Arrays :

```
let arr: number[] = [1, 2, 3];
let arr2: number[][] = [
  [1, 2, 3],
  [1, 2, 3],
];
```

Annotating objects :

We can annotate object in three ways :

1. Using object Literal -> {}
2. Using Javascript constructor -> Object
3. Using typescript non-primitive type -> object

{}	Object	object
object literal	javascript object constructor	typescript object type
accepts both primitives and non-primitives	accepts both primitives and non-primitives	accepts only non-primitives
accepts any non-nullish values	accepts any non-nullish values	accepts any non-nullish values

Using Object Literal :

```
let i: {};
i = 10;
i = "hello";
i = true;
i = [];
i = {};
i = function add() {};
i = null;
i = undefined;
```

Using Javascript Object Constructor :

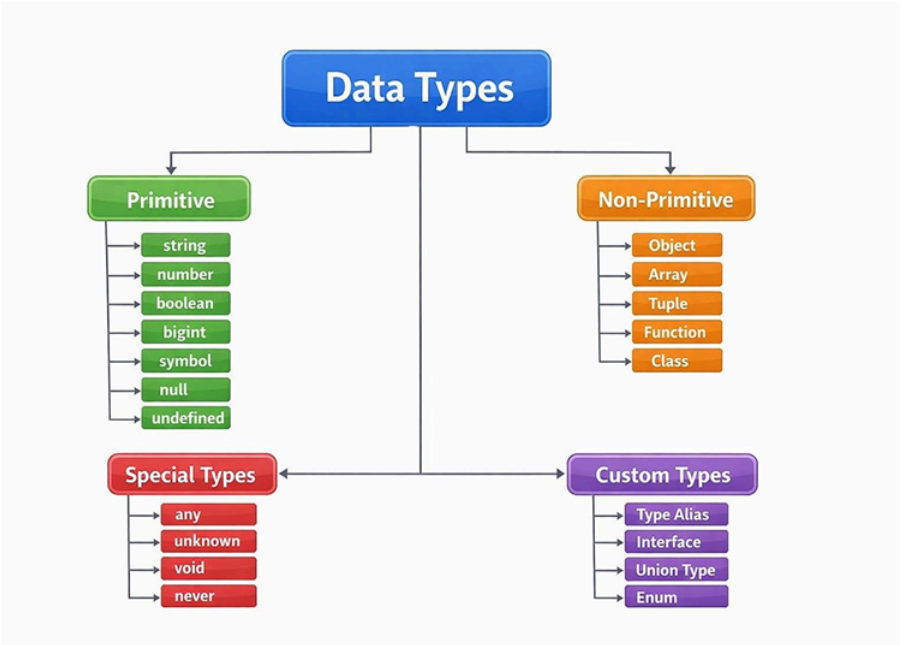
```
let j: Object;
j = 10;
j = "string";
j = true;
j = [];
j = {};
j = function demo() {};
j = null;
j = undefined;
```

Using typescript object type :

```
let k: object;
k = 10;
k = "hello";
k = true;
k = [];
k = {};
k = function demo() {};
k = null;
k = undefined;
```

Data Types :

- A data type specifies the kind of value a variable can store and the operations that can be performed on it.
- In TypeScript, data types ensure type safety, correct operations, and reduced runtime errors.



Type Alias :

- A type alias is a way to give a custom name to a type.
- It helps make complex types readable, reusable, and maintainable.

Syntax :
type AliasName = Type;

Type alias for Primitives :

```
1   type num = number;
2   type str = string;
3   type bool = boolean;
4   type sym = symbol;
5   type nul = null;
6   type unD = undefined;
```

Type alias for arrays :

```
131    type num = number[];
132    type skills2 = string[];
133    type nested = number[][];
```

Type Alias for Objects :

```
77    type API = {
78      id: number;
79      name: string;
80      bloodgroup: string;
81      place: string;
82    };
```

Optional Properties (?)

An optional property is an object property that may or may not exist. It is marked using a question mark ? after the property name.

Syntax :
type User = {
name: string;
age?: number; // optional property
};

Example :

```
140   type user = {
141     name: string;
142     dob: string;
143     id: number;
144     skills?: skills2; //? optional property
145   };
```

Tuple in TypeScript

A tuple is a fixed-length array where:

- the order of elements matters
- each position has a specific type

Tuples are stricter than arrays

Syntax :
let tupleName: [type1, type2];

Example :

```
61    let arr2: [number, string, boolean] = [101, "Romeo", true];
62    let arr3: [number[], string[], boolean[]] = [
63      [1, 2, 3],
64      ["Romeo", "devadas"],
65      [true, false],
66    ];
```

Tuple with Optional Element :

```
let employee: [number, string, string?] = [1, "Ravi"];
```

Tuple in Function return type:

```
function getUser(): [number, string] {
return [1, "Admin"];
}
```

Tuple in Function Parameter :

```
function printUser(user: [number, string]): void {
console.log(user[0], user[1]);
}
```

Union Types in TypeScript

- A union type allows a variable to hold one of multiple specified types.
- Use | (pipe symbol).

Syntax :

```
 let <varialbeName> : type1 | type2 | type3 | .. typeN
```

Examples :

```
192    let m: number | string;
193    m = "hello";
194    m = 10;
195    m = "hi";
196    m = 876545;
```

Unions with functions :

```
198  function demo2(id: number | string, isAvailable: string | boolean): string | boolean {
199    return `this is id : ${id} and he is ${isAvailable ? "available" : "not available"}`;
200  }
```

Unions with Objects :

```
type student = {
  name: string;
  marks: number;
};
type teacher = {
  name: string;
  subject: string;
};

type person = student | teacher;
let p: person = {
  name: "nidhi",
  marks: 100,
};
```

Discriminated Unions or Tagged Uniones :

A union of object types that share a common field with unique literal values, used to safely determine the exact type.

A **discriminated union** is a pattern in TypeScript where:

1. You combine multiple object types
2. Each type has a **common property (called discriminator)**
3. That property has **different literal values**
4. TypeScript uses it to **automatically narrow the type**

Example :

```
type placed = { status: string; orderTime: string };
type preparing = { status: string; chefName: string };
```

```
type delivered = { status: string; deliveryTime: string };
type order = placed | preparing | delivered;
let order1: order = {
  status: "placed",
  orderTime: "7:30PM",
};
let order2: order = {
  status: "preparing",
  chefName: "rangnatha",
};
let order3: order = {
  status: "delivered",
  deliveryTime: "9:30PM",
};
function handleOrder(order: order) {
  if ("orderTime" in order) {
    console.log("the order is placed at : ", order.orderTime);
  } else if ("chefName" in order) {
    console.log("the order is being prepared by : ", order.chefName);
  } else if ("deliveryTime" in order) {
    console.log("the order is delivered at : ", order.deliveryTime);
  } else {
    console.log("status is invalie");
  }
}
handleOrder(order1);
handleOrder(order2);
handleOrder(order3);
```

Intersection Types in TypeScript :

- An intersection type combines multiple types into one.
- The resulting type must satisfy all the combined types. Uses & (ampersand)

Syntax :
type NewType = TypeA & TypeB;

Example :

```
type Person = {
  name: string;
  age: number;
  bloodGroup: string;
};
type Emp = {
```

```
  id: number;
  empno: number;
  doj: string;
};

type staff = Person & Emp;
let details: staff = {
  name: "romeo",
  age: 25,
  bloodGroup: "A+",
  id: 101,
  empno: 7799,
  doj: "january/01/2015",
};
```

Intersection with Union Types :

```
type Admin = { role: string };
type User = { role: string };
type idNumber = { id: number };
type account = (Admin | User) & idNumber;
let userAccout: account = {
  role: "admin",
  id: 101,
};
```

Enums in TypeScript

An enum (enumeration) is a special type in TypeScript used to define a set of named constants. It makes code more readable and meaningful.

Syntax :

```
enum <name>{ set of constants }
```

Types of Enums :

1. Numeric enum
2. String enum
3. Computed enum
4. Mixed enum

Numeric Enum :

- A numeric enum stores number values.
- If values are not assigned, it automatically starts from 0 and increments.
- Supports reverse mapping
- Auto-increment works

Example :

```
enum Role {
```

```
  Admin, // 0
  User, // 1
}
// this is for Role enum
let role = Role.Admin;
let role2 = Role.User;
```

String enum :

- A string enum stores string values.
- Each member must be initialized with a string literal.
- No reverse mapping
- More readable in APIs

Example :

```
enum str {
  name = "john",
  address = "Germany",
  mail = “john@gmail.com”,
}
```

Mixed Enum :

- A mixed enum contains both numeric and string values.
- Allowed but not recommended in real projects.
- Avoid using mixed enums in production code.

Example :

```
enum mix {
  a = 1,
  b = 3,
  c = "Admin",
  d = "user",
}
```

Computed Enum :

- A computed enum contains values calculated using expressions.
- After a computed value, automatic numbering is not allowed.
- Must manually assign values after computed members.

```
enum com {
  a = 1,
  b = 10,
  c = 19,
  d = a + b + c,
}
```

Special Type :

- A Special Type in TypeScript is a built-in type that is designed to handle special situations in type checking — such as unknown values, functions that do not return, or disabling type safety.
- These types do not describe normal data like string or number. Instead, they control how TypeScript behaves during type checking.

List of Special types :

1. Any
2. Unknown
3. Void
4. Never

any :

- The any type allows a variable to hold any type of value.
- It disables type checking for that variable.

Characteristics :

- Can store values of any type
- TypeScript does not perform type checking
- All operations are allowed

When to Use :

- During migration from JavaScript to TypeScript
- When working with third-party libraries without type definitions
- When the type is completely dynamic

Important Note :
Overusing any removes the main advantage of TypeScript, which is type safety.

Example :

```
let q: any;
q = 10;
q = "";
q = {};
q = [];
q = true;
```

Unknown :

- The unknown type represents a value whose type is not known at the time of writing code.
- It is a safer alternative to any.

Characteristics :

- Can store any value
- Cannot perform operations without type checking
- Requires type narrowing before usage

Example :

```
let t: unknown;
t = 10;
t = "hello";
t = {};
t = true;
t = [];
// typeGuard
if (typeof t === "number") {
  t.toString();
}
```

Difference Between any and unknown

Feature	any	unknown
Accepts any value	Yes	Yes
Type checking required	No	Yes
Safe for large projects	No	Yes

Void :

- The void type is used when a function does not return any value.

Characteristics :

- Mainly used as a function return type
- Indicates that nothing is returned

Note :

A function with void return type may return undefined, but it should not return any meaningful value.

Example :

```
function logMessage(message: string): void {
  console.log(message);
}
```

Never :

- The never type represents values that never occur.
- It is used when a function never completes normally

Situations Where never Is Used

- Function throws an error
- Infinite loop
- Exhaustive type checking

Purpose

- Ensures all possible cases are handled
- Helps detect logical errors during development

Example :

```
type lTypes = "success" | "pending" | "faild";
function test(ty: lTypes) {
  if (ty === "success") {
    console.log("the operation is done", ty);
  } else if (ty === "pending") {
    console.log("it is still in pending", ty);
  } else if (ty === "faild") {
    console.log("the operation is faild", ty);
  } else {
    console.log(ty);
  }
}
```

Type Assertion in TypeScript :

Type Assertion is a feature in TypeScript that allows a developer to manually specify the type of a value when the compiler cannot automatically infer it correctly.

Why Do We Need Type Assertion?

- Sometimes TypeScript cannot correctly infer the type.
- In such cases, we manually specify the type.

Syntax :

```
// 1. Angle Bracket Syntax
let value: unknown = "Hello";
let strLength: number = (<string>value).length;
// 2. as Syntax (Recommended)
let value: unknown = "Hello";
let strLength: number = (value as string).length;
The as syntax is preferred, especially in React projects.
```

Example :

```
let input = document.getElementById("username") as HTMLInputElement;
input.value = "Vishnu";
//Here we assert that the element is an HTMLInputElement.
```

Important Notes

- Type assertion does not perform runtime checking.
- It does not convert the type.
- It only affects compile-time behavior.

Literal Types in TypeScript :

- Literal Types are types that represent exact values instead of general types.

String Literals :

- Used commonly in APIs and UI states.

Example :

```
let status: "success" | "error" | "loading";
status = "success";   // allowed
```

Number Literal Types :

Example :

```
let rating: 1 | 2 | 3 | 4 | 5;
rating = 5;  // allowed
```

Why Use Literal Types?

- Prevent invalid values
- Improve type safety
- Make APIs strict and predictable
- Useful with union types

Interface in TypeScript

- An Interface in TypeScript is a blueprint used to define the structure of an object. It specifies: Property names
- Property types
- Method signatures It does not contain implementation.
- It only defines the shape of data.

Syntax :

```
syntax : interface <interface>{ // block of code }
```

Example :

```
interface demo {
  name: string;
  age: number;
  address: string;
  skills: string[];
}
interface demo {
  empNo: number;
  empID: string;
  salary: number;
  role: string;
  dept: string;
}

let interData: demo[] = [
  {
    name: "roger",
    age: 45,
    address: "longTown",
    skills: ["sword fight"],
    empNo: 101,
    empID: "tyss",
    salary: 98765432,
    role: "developer",
    dept: "IT",
  },
  {
    name: "roger",
    age: 45,
    address: "longTown",
    skills: ["sword fight"],
```

```
    empNo: 101,
    empID: "tyss",
    salary: 98765432,
    role: "developer",
    dept: "IT",
  },
}
```

Classes in TypeScript

- A class is a blueprint used to create objects with properties and methods.
- It helps implement object-oriented programming concepts in TypeScript.

Syntax :
class ClassName {
property: type;
constructor(parameter: type) {
this.property = parameter;
}
method(): returnType {}
}

Example :

```
class car {
  brandName: string;
  price: number;
  tax: number;
  engine: string;
  tires: number;
  constructor(
    brandName: string,
    price: number,
    engine: string,
    tires: number,
    tax: number,
  ) {
    this.brandName = brandName;
    this.price = price;
    this.engine = engine;
    this.tires = tires;
    this.tax = tax;
  }
  drive() {
```

```
    return `the car name is ${this.brandName} and its price
${this.price}which is haveing ${this.engine}`;
  }
  behaviour() {
    return `the total amount of the product is : ${this.price +
this.tax}`;
  }
}
let object = new car("BMW", 123453, "V8", 4, 20);
console.log(object.brandName);
console.log(object.price);
console.log(object.engine);
console.log(object.tires);
```

Constructor :

- A constructor is a special method used to initialize class properties.
- It runs automatically when an object is created.

Syntax :

constructor(parameters) {
// initialization
}

Example :
class Car {
brand: string;
constructor(brand: string) {
this.brand = brand;
}
}

Inheritance :

- Inheritance is a mechanism where one class (child) **reuses properties and methods** of another class (parent).
- Instead of rewriting logic, you **extend behavior**.

To achieve inheritance we need to use :

1. Extends : `extends` is used when one class **inherits from another class**.
    - class to class.
    - Interface to interface

Syntax for extends :

```
class Parent {
  // properties & methods
}

class Child extends Parent {
  // additional properties & methods
}
```

Example :

```
class Animal {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  makeSound() {
    console.log("Some sound");
  }
}

class Dog extends Animal {
  bark() {
    console.log(this.name + " barks");
  }
}
let dog = new Dog("Tommy");
dog.makeSound(); // inherited
dog.bark(); // own method
```

2. Implements : `implements` is used when a class **agrees to follow a contract (interface)**.
    - class to interface

Syntax for implements :

```
interface InterfaceName {
  property: type;
  method(): void;
}
```

```
class MyClass implements InterfaceName {
  // must implement everything
}
```

Example :

```
interface Vehicle {
  speed: number;
  drive(): void;
}

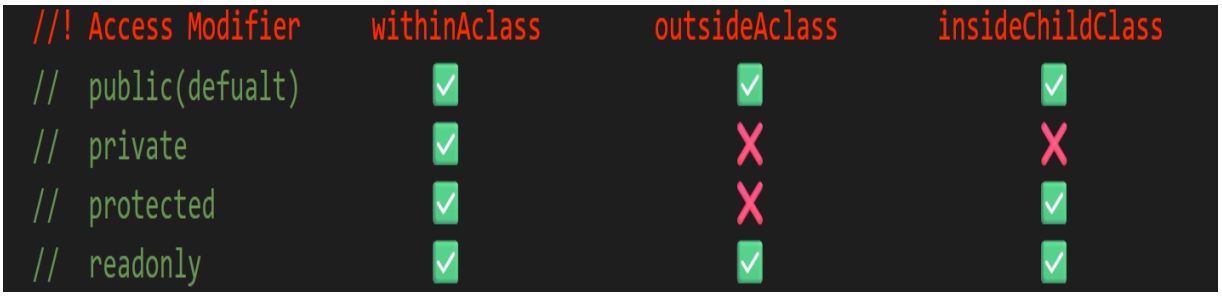
class Car implements Vehicle {
  speed: number;

  constructor(speed: number) {
    this.speed = speed;
  }

  drive() {
    console.log("Car is driving at " + this.speed);
  }
}

const car = new Car(100);
car.drive();
```

ACCESS MODIFIERS IN TYPESCRIPT :
Access modifiers control **visibility (access level)** of properties and methods in a class.



1. `public` (Default)

Accessible **everywhere** — inside class, outside class, child classes.

Example :

```
class Person {
  public name: string;

  constructor(name: string) {
    this.name = name;
  }
}

const p = new Person("Pavan");
console.log(p.name); // ✅ accessible
```

2. `private`

Accessible **ONLY inside the same class**.

Example :

```
class BankAccount {
  private balance: number;

  constructor(balance: number) {
    this.balance = balance;
  }

  showBalance() {
    console.log(this.balance);
  }
}

const acc = new BankAccount(1000);

console.log(acc.balance); // ❌ ERROR
```

3. `protected`

Accessible:

- ✅ Inside class
- ✅ Inside child classes
- ❌ NOT outside

Example :

```
class Animal {

  protected name: string;



  constructor(name: string) {

    this.name = name;

  }

}

class Dog extends Animal {

  speak() {

    console.log(this.name); // ✅ allowed

  }

}

const d = new Dog("Tommy");

console.log(d.name); // ❌ ERROR
```

Readonly :

`readonly` is used to make a property **immutable after initialization**.

Example :

```
class User {
  readonly id: number;

  constructor(id: number) {
    this.id = id;
  }
}

const u = new User(101);

u.id = 202; // ❌ ERROR: Cannot assign to 'id'
```

ENCAPSULATION

Encapsulation is the process of **wrapping data (variables) and methods into a single unit (class)** and **restricting direct access to the data**.

- **Data Hiding**
  Prevent direct access to variables using `private`
- **Controlled Access**
  Provide access using **public methods (getters & setters)**

Syntax :

```
class ClassName {
  private property: type;

  constructor(value: type) {
    this.property = value;
  }

  // Getter
  getProperty() {
    return this.property;
  }

  // Setter
  setProperty(value: type) {
    this.property = value;
  }
}
```

Example :

```
class atm2 {
  private balance: number;
  private pin: string;
  private statements: string;
  constructor(balance: number, pin: string, statement: string) {
    this.balance = balance;
    this.pin = pin;
    this.statements = statement;
  }
  // getter
  getBalance() {
    return this.balance;
  }
```

```
  getStatements() {
    return this.statements;
  }
  //setter
  setBalance(updatedBalance: number) {
    this.balance = updatedBalance;
  }
  setStatements(updatedStatement: string) {
    this.statements = updatedStatement;
  }
}
```

METHOD OVERLOADING

Method overloading means **same method name, different parameters (type or number)**.

There is NO true runtime overloading — it's handled using multiple signatures + one implementation

Example :

```
class Calculator {
  add(a: number, b: number): number;
  add(a: string, b: string): string;

  add(a: any, b: any): any {
    return a + b;
  }
}

const c = new Calculator();

console.log(c.add(2, 3));        // 5
console.log(c.add("Hi ", "JS")); // Hi JS
```

METHOD OVERRIDING

Method overriding means **child class provides its own implementation of a parent class method**.

Example :

```
class Animal {
  makeSound(): void {
    console.log("Animal makes sound");
  }
}

class Dog extends Animal {
  makeSound(): void {
    console.log("Dog barks");
  }
}

const d = new Dog();
d.makeSound(); // Dog barks
```

STATIC KEYWORD IN TYPESCRIPT

`static` is used to define properties or methods that belong to the **class itself**, not to individual objects.

- You don't need to create an object to use it

Example :

```
class college {
  static collegeName: string = "ABC college";
  static add() {
    return "this is add";
  }
}

let c1 = new college();
console.log(c1.collegeName); //❌Error

console.log(college.collegeName);
console.log(college.add());
```

ABSTRACTION IN TYPESCRIPT

Abstraction means **hiding implementation details and showing only essential features**.

How to Achieve in TypeScript

You can achieve abstraction using:

1. **Abstract Classes**
2. **Interfaces**

1. ABSTRACT CLASS

An abstract class is a class that:

- Cannot be instantiated
- Can have:
    - abstract methods (no body)
    - normal methods (with body)

Example :

```
abstract class Vehicle {
  abstract start(): void;

  stop() {
    console.log("Vehicle stopped");
  }
}

class Car extends Vehicle {
  start(): void {
    console.log("Car starts with key");
  }
}

const c = new Car();
c.start();
c.stop();
```

USING INTERFACE (ABSTRACTION)

Interface defines a **contract (only structure)** — no implementation.

Example :

```
interface Shape {
  area(): number;
}

class Circle implements Shape {
  radius: number;

  constructor(radius: number) {
    this.radius = radius;
  }

  area(): number {
    return Math.PI * this.radius * this.radius;
  }
}
```

Generics in TypeScript

- Generics allow us to create reusable components that work with different data types.
- They provide flexibility while maintaining strong type safety.

Why We Need Generics

Without generics, we either:

- Use a specific type (less reusable), or
- Use any (loses type safety).

Generics solve both problems by allowing dynamic types with safety.

Generic Function :
Syntax :

```
function functionName<T>(parameter: T): T {
return parameter;
}
```

Example :

```
function identity<T>(value: T): T {
return value;
}
let num = identity<number>(10);
let str = identity<string>("Hello");
// Here, T is a type parameter.
```

Generic with Multiple Types :

Syntax :

```
function functionName<T, U>(param1: T, param2: U): void {}
```

Example :

```
function combine<T, U>(a: T, b: U): string {
return `${a} ${b}`;
}
combine<number, string>(10, "Apples");
```

Generic Interfaces

Syntax :

```
interface InterfaceName { property: T }
```

Example :

```
interface Box<T> {
value: T;
}
let numberBox: Box<number> = { value: 100 };
let stringBox: Box<string> = { value: "Hello" };
```

Generics in classes

Syntax :

```
class ClassName<T> {
property: T;
constructor(value: T) {
this.property = value;
}
}
```

Example :

```
class DataStore<T> {
  data: T;
  constructor(data: T) {
    this.data = data;
  }
  getData(): T {
    return this.data;
  }
}
let store = new DataStore<string>("TypeScript");
```

Decorators in Typescript :

Decorators are special functions used to add metadata or modify behavior of classes, methods, properties, or parameters.

- They are written using @ symbol
- Applied **before** the target

Simple Understanding

Adding extra behavior without changing original code

Example:

- You wrap a function with logging
- You don’t modify original function
- You enhance it externally

Syntax :

```
function decoratorName(target: any) {
  // logic
}

@decoratorName
class MyClass {}
```

Example :

```
function Logger(constructor: Function) {
  console.log("Class created:", constructor.name);
}

@Logger
class User {}
```